Self-Deploy — Microsoft Entra: Identity Fundamentals

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Self-Deploy: Microsoft Entra Identity Fundamentals 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

What this delivers . . . 6

The standard deployment model . . . 6

Outcome . . . 6

Important boundaries . . . 6

1. Solution Overview — Self-Deploy: Microsoft Entra Identity Fundamentals 6

What this is . . . 6

How the technology works (the mental model) . . . 7

The standard deployment model . . . 8

Scope — what this kit covers . . . 8

Out of scope (handled elsewhere) . . . 8

Benefits . . . 9

Limitations (be honest with buyers) . . . 9

Authentication and connections used . . . 9

2. Requirements Guide — Self-Deploy: Microsoft Entra Identity Fundamentals 10

R1. Portal access to the Microsoft Entra admin center . . . 10

R2. A learner cohort and least-privilege lab roles . . . 10

R3. A safe, cloud-only lab sandbox posture . . . 11

R4. Break-glass accounts protected (before any Conditional Access lab) . . . 11

R5. Entra ID licensing awareness (P1/P2 and add-ons) . . . 11

R6. The PowerShell + KQL helper toolchain . . . 12

Checklist . . . 12

Global Requirements — Entra ID Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 12

G2. Entra ID licensing (P1, P2 where noted) . . . 13

G3. Administrative roles . . . 13

G4. Break-glass (emergency access) accounts . . . 13

G5. Healthy on-premises Active Directory . . . 13

G6. An Entra Connect host server . . . 13

G7. UPN / domain hygiene . . . 13

G8. Network access to Microsoft cloud endpoints . . . 13

3. Implementation Guide — Self-Deploy: Microsoft Entra Identity Fundamentals 14

Phase 0 — Assess (read-only, changes nothing) . . . 15
Phase 1 — Configure (stand up the cloud-only lab sandbox) . . . 15
Phase 2 — Pilot (the guided labs) . . . 15
Phase 3 — Validate (capstone + knowledge-check) . . . 17
Phase 4 — Expand (operate with confidence) . . . 17
Rollback / safe-off . . . 18
5. Testing & Validation — Self-Deploy: Microsoft Entra Identity Fundamentals 18
Gate 1 — The tenant is reachable and the fundamentals picture is captured . . . 18
Gate 2 — The learner cohort exists . . . 19
Gate 3 — The cloud-only lab sandbox exists and is correctly named . . . 19
Gate 4 — Learners can read a user, group, role, and app unaided . . . 19
Gate 5 — The sign-in labs run and the CA lab policy is report-only . . . 19
Gate 6 — Break-glass is excluded from the CA lab policy . . . 20
Gate 7 — The knowledge-check passes . . . 20
Gate 8 — Read-only-first posture held; roles least-privilege . . . 20
6. Troubleshooting — Self-Deploy: Microsoft Entra Identity Fundamentals 21
1. A learner cannot open a blade / everything is greyed out . . . 21
2. `Connect-MgGraph` fails or a script returns 403 . . . 21
3. The lab sandbox setup won't create objects . . . 22
4. A lab test user cannot be edited (it's synced) . . . 22
5. Dynamic-group membership didn't update . . . 22
6. The Conditional Access lab policy isn't report-only (or can't be built) . . . 22
7. Break-glass is not excluded from the CA lab policy . . . 23
8. Sign-in logs show nothing / the KQL lab returns no rows . . . 23
9. "Which license do I have?" confusion . . . 23
10. App registration vs enterprise application confusion . . . 23
7. Best Practices — Self-Deploy: Microsoft Entra Identity Fundamentals 23
Learn on a sandbox, never on production . . . 24
Treat the identity plane as high-risk . . . 24
Build the right mental model . . . 24
Verify, don't assume . . . 24
Run it as a program, not a one-off class . . . 25
Where to go next . . . 25
8. Deliverables Summary — Self-Deploy: Microsoft Entra Identity Fundamentals 25
Documentation . . . 25
Lab helpers (5) . . . 26

Named objects (verbatim — used across docs, scripts, and tests) 27
What this kit deliberately does NOT do 28
How it is delivered 28
Outcome 28
Appendix A — PowerShell Scripts 29

Executive Summary — Self-Deploy: Microsoft Entra Identity Fundamentals

White-label · Entra ID Solutions

The problem

A team inherits a Microsoft Entra tenant — users are already there, groups exist, someone set up sign-in and some policies — but nobody on the team can confidently say how any of it works. New admins are handed the keys to the identity plane, the single most dangerous surface in the whole Microsoft cloud, without a mental model of tenants, users, groups, roles/RBAC, app registrations, sign-in, MFA, or Conditional Access. The result is predictable: risky changes made by guesswork, tickets that sit because no one knows where the setting lives, and a reluctance to touch anything for fear of locking the company out of its own directory.

The solution

This is a **self-guided self-deploy** — concepts, **guided hands-on labs**, a **take-home runbook and glossary**, and a **knowledge-check** — that takes a novice-but-competent admin from zero to a solid working grounding in Microsoft Entra identity fundamentals. It is deliberately a **skills** product: the outcome is people who understand the tenant and can operate it safely, not another policy someone else configured.

- **Teaches the model, not just the clicks** — what a tenant is, how users and groups and roles actually relate, what an app registration versus an enterprise application is, how a sign-in is evaluated, and where MFA and Conditional Access fit.
- **Ships guided, read-only-first labs** — every lab runs in a small, named **lab sandbox** with cloud-only test objects, so learners practice on real Entra without endangering production.
- **Ends in durable artifacts** — a take-home **fundamentals runbook**, a **glossary**, and a scored **knowledge-check** so you can prove the skills landed.

What you get

- A plain-language **Solution Overview** of the Entra model — tenant, identities, groups, roles and RBAC, app registrations vs enterprise apps, the sign-in flow, MFA methods, Conditional Access basics, and the licensing / admin-role picture.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (on top of the shared category requirements).
- An **Implementation Guide** structured as guided labs — build the cohort and lab sandbox, then work through users/groups, roles/RBAC, app registrations, sign-in/MFA, and a **report-only** Conditional Access lab, then a capstone and knowledge-check.
- **Five lab helpers** — a read-only tenant-fundamentals discovery, a gated lab-sandbox setup

(preview by default), a sign-in/audit **KQL lab pack**, a read-only validation/knowledge-check runner, and a cohort/fundamentals report.

- **Testing and Troubleshooting** guides so the team learns to verify and self-diagnose.

What this delivers

New admins with a real working grounding in Microsoft Entra — able to read the directory, manage users and groups, understand least-privilege admin roles, recognize an app registration, follow a sign-in through MFA and Conditional Access, and make safe changes — plus a take-home runbook, glossary, and a passed knowledge-check that proves the capability is there.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand — reframed for enablement: assess readiness, configure the cohort and lab sandbox, run the guided labs (pilot), validate skills with a capstone and knowledge-check, then expand into confident day-to-day operation and the advanced solutions.

Outcome

A team grounded in Microsoft Entra fundamentals — ready to operate and secure the tenant and to progress to the advanced identity solutions (Conditional Access, PIM, passwordless, governance) without being lost.

Important boundaries

This is a **skills / enablement** product — it teaches people the fundamentals and lets them practice in a safe sandbox. It **cross-references, and deliberately does not duplicate**, the catalog's **implementation** solutions — Conditional Access baseline (ENTZTCA), Conditional Access Integration, Privileged Identity Management (ENTPIM), Passwordless & Passkey (ENTPASSKEY), Identity Governance & Access Reviews (ENTIGA), Hybrid Identity, SAML App Factory, and Enterprise-App governance — which *deploy and operate* those capabilities. This kit builds the grounding; those products build the production systems. Any labs that touch Conditional Access run **report-only first**; break-glass accounts are protected throughout. Where a feature needs a specific license (Entra ID P1/P2, Identity Governance, Global Secure Access, Verified ID) or is in preview, this kit flags it — *verify current availability and licensing in your tenant*, which can differ by cloud (Commercial vs GCC / GCC High).

1. Solution Overview — Self-Deploy: Microsoft Entra Identity Fundamentals

What this is

A complete, self-guided **self-deploy** that takes a novice-but-competent admin from zero to a solid working grounding in **Microsoft Entra identity fundamentals** — tenants, users, groups, roles and RBAC, app registrations, the sign-in flow, MFA, and Conditional Access basics — plus the licensing and admin-role model. It is delivered as concepts, **guided hands-on labs**, a **take-home runbook and glossary**, and a **knowledge-check**.

It is a **skills** product — the outcome is people who can read and operate the tenant. It is **adjacent to** the catalog's implementation solutions (Conditional Access baseline, PIM, passwordless, governance, hybrid identity); those *deploy and operate* the advanced capabilities, while this kit builds the fundamentals that make them make sense. The two are complementary and are cross-referenced, not duplicated.

How the technology works (the mental model)

- **The tenant is your identity boundary.** A **Microsoft Entra tenant** is a dedicated, isolated

instance of the directory that Microsoft creates for your organization. It has an initial domain (`contoso.onmicrosoft.com`), zero or more verified custom domains (`contoso.com`), a tenant ID, and it holds every identity and configuration below. Everything else in this kit lives inside one tenant.

- **Identities are who can sign in.** Entra holds several kinds of **identity**: member **users**

(your staff), **guest** users (external people invited via B2B), and **service principals / workload identities** (apps and automation, not people). Each user has a **User Principal Name** (UPN, e.g. `ana@contoso.com`), an object ID, and attributes.

- **Groups are how you manage people in bulk.** A **security group** grants access or targets policy

to many users at once; a **Microsoft 365 group** additionally backs a mailbox/Team. Membership is either **assigned** (you add people) or **dynamic** (a rule such as `department eq "Sales"` adds them automatically). You target licenses, app access, and Conditional Access at groups, not individuals.

- **Roles + RBAC decide who can administer what.** Entra has built-in **directory roles** (Global

Administrator, User Administrator, Helpdesk Administrator, Security Reader, and dozens more). A role is a bundle of permissions; assigning it to a user grants those permissions, ideally **scoped** and **least-privilege**. Global Administrator is all-powerful and should be rare; most day-to-day work needs a narrow role. (Azure RBAC on subscriptions/resources is a separate, related system — this kit focuses on the Entra directory roles.)

- **Apps come in two shapes.** An **app registration** is the *definition* of an application in your

tenant (its identity, redirect URIs, API permissions, secrets/certificates). An **enterprise application** (service principal) is the *instance* of an app — yours or a third party's — that users actually sign in to and that you assign, consent, and govern. Understanding the registration vs enterprise-app distinction is a fundamental most new admins are missing.

- **Sign-in is an evaluated flow.** When a user signs in, Entra authenticates them (password,

passwordless, or federated), then evaluates **Conditional Access** — an if-then engine that can require MFA, a compliant device, or block the sign-in based on who/what/where/risk — before issuing a token. **MFA** (Multi-Factor Authentication) adds a second factor; methods range from SMS (weak) to authenticator apps to **phishing-resistant** FIDO2/passkeys (strong).

- **Licensing gates the features.** Core directory, users, groups, and self-service basics are

included with Microsoft 365. **Entra ID P1** adds Conditional Access, group-based licensing, and more; **Entra ID P2** adds Identity Protection risk policies and Privileged Identity Management (PIM). Identity Governance, Global Secure Access, and Verified ID are further add-ons. *Verify what your tenant is licensed for — bundles change, and availability differs by cloud.*

The standard deployment model

Every solution in this program follows the same five stages; here they structure the kit:

1. **Assess.** Confirm the learners and tenant are ready — an admin can open the Entra admin center, the operator holds least-privilege roles, and a safe lab posture is agreed (run `EKIDFUND-Discovery.ps1` for a read-only picture of the tenant fundamentals).

1. **Configure.** Create the learner cohort group `Grp-EntraFund-Cohort` and a small, cloud-only **lab sandbox** (`EKIDFUND-LabSetup.ps1`, preview by default) so labs never touch production objects.

1. **Pilot.** Run the **guided labs** (Labs 1–6) with the cohort — tenant tour, users, groups & dynamic membership, roles/RBAC, app registrations, and a **report-only** sign-in/MFA/Conditional Access lab — all read-only-first.

1. **Validate.** Run the **capstone** exercise and score the **knowledge-check** `Check-EntraFund-Pilot` (`EKIDFUND-Validate.ps1`).

1. **Expand.** Fold the grounding into confident day-to-day operation, tear down or keep the sandbox, and progress to the advanced identity solutions.

Scope — what this kit covers

• **Tenant fundamentals** — what a tenant is, domains, tenant ID, the Entra admin center layout.

• **Identities** — member users, guests (B2B basics), service principals/workload identities, UPNs and attributes.

• **Groups** — security vs M365 groups, assigned vs dynamic membership, group-based targeting.

• **Roles & RBAC** — built-in directory roles, least privilege, Global Administrator hygiene, and how to read `Roles and administrators`.

• **App registrations vs enterprise applications** — the definition vs the instance, API permissions and consent at a conceptual level.

• **Sign-in, MFA & Conditional Access basics** — the sign-in flow, MFA methods (including phishing-resistant), and a first, safe, **report-only** Conditional Access policy.

• **Licensing & admin-role model overview** — what P1/P2 and the add-ons unlock.

• **A take-home runbook, glossary, and knowledge-check** — so the skills persist and can be proven.

Out of scope (handled elsewhere)

• **Production Conditional Access design and rollout.** Building a full CA baseline is the **Zero Trust Conditional Access Baseline (ENTZTCA) **and** Conditional Access Integration** solutions. This kit builds *one report-only lab policy* to teach the concept; it does not design a production baseline.

• **Privileged access, passwordless, governance, hybrid.** Just-in-time admin (PIM), FIDO2/passkey rollout, access reviews/entitlement management, and AD-to-Entra sync are the **ENTPIM**, **ENTPASSKEY**, **ENTIGA**, and **Hybrid Identity** solutions — introduced here, delivered there.

• **App publishing at scale.** Standing up SAML/OIDC apps and governing the enterprise-app estate are the **SAML App Factory** and **Enterprise-App & Service-Principal Governance** solutions.

• **The business/security determinations.** Which roles are "privileged," what MFA methods are mandated, which apps are sanctioned, and every production change are the customer's security/operations decisions; this kit provides the skill and the labs, not the decisions.

Benefits

• **Confidence** — new admins understand what they are looking at and can make safe changes instead of guessing.

• **Safety** — labs run in a cloud-only sandbox and any Conditional Access lab is **report-only**, so learning never risks a lock-out.

• **Repeatability** — a take-home runbook, glossary, and knowledge-check make onboarding the next new hire a repeatable exercise.

• **A foundation that compounds** — the grounding makes every advanced identity solution in the catalog land faster.

Limitations (be honest with buyers)

• This teaches **fundamentals**; it is not a production configuration. The one CA lab policy is a teaching aid in report-only, not a baseline.

• Some concepts referenced (P2 risk policies, PIM, Identity Governance, Global Secure Access, Verified ID) need specific licensing or are in preview — *verify availability in your tenant*.

• Feature names, portal navigation, and licensing bundles evolve and can differ by cloud (Commercial vs GCC / GCC High) — *verify against your tenant*.

Authentication and connections used

• **Microsoft Entra admin center** (`https://entra.microsoft.com`) for the interactive labs — users, groups, roles, app registrations, sign-in logs, and Conditional Access.

• **Microsoft Graph PowerShell SDK** for the read-only discovery/validation/reporting helpers and the gated, preview-by-default lab-sandbox setup (each script's header names its least-privilege scopes).

• **KQL** over Entra **sign-in and audit logs** (via the Graph audit-log endpoints, or a Log Analytics workspace where sign-in logs are exported) for the sign-in/MFA labs.

• Helpers authenticate **interactively** with an MFA-capable account; **no credentials are stored in

code**. Government-cloud tenants pass the matching endpoints — *verify endpoints for your cloud*.

2. Requirements Guide — Self-Deploy: Microsoft Entra Identity Fundamentals

These are the **solution-specific** prerequisites for running the Identity Fundamentals enablement kit. The **category baseline** — an active Microsoft Entra tenant, Entra licensing, least-privilege admin roles, break-glass accounts, and network access — is covered once in `global-requirements-entra-id-solutions.md` (**G1–G8**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. Portal access to the Microsoft Entra admin center

• **Description:** Learners can open `https://entra.microsoft.com` and see the directory — Overview, Users, Groups, Roles and administrators, Applications, and Protection (Conditional Access).

• **Why required:** The interactive labs are performed in this portal; without access there is nothing to explore.

• **How to verify:** Sign in and confirm the **Overview** blade loads and shows the tenant name and tenant ID. Or run `EKIDFUND-Discovery.ps1`, which confirms the directory is reachable read-only.

• **How to implement if missing:** Grant each learner at least a read role (e.g. **Global Reader**); confirm they can reach the portal from their workstation (network access is **G8**).

R2. A learner cohort and least-privilege lab roles

• **Description:** A named group of learners — `Grp-EntraFund-Cohort` — each holding a **read** role for exploration (**Global Reader** or **Directory Readers**). The instructor holds the higher roles needed to *create* the lab sandbox objects (**User Administrator** + **Groups Administrator** + **Application Administrator**, or **Global Administrator** for the one-time setup).

• **Why required:** Labs run at **least privilege** — learners read and follow along, and only the instructor creates the sandbox objects. This models the real least-privilege discipline the kit teaches.

• **How to verify:** Entra -> **Roles and administrators**; confirm the cohort holds read roles and the instructor holds the setup roles. `EKIDFUND-Discovery.ps1` lists directory-role assignments.

• **How to implement if missing:** Create `Grp-EntraFund-Cohort` in Entra (or with `EKIDFUND-LabSetup.ps1`); assign read roles to learners; reserve create/change roles to the instructor (prefer **PIM-eligible** where Entra ID P2 is available — *verify licensing*).

R3. A safe, cloud-only lab sandbox posture

• **Description:** An agreed rule that all labs are performed on **cloud-only, clearly named test objects** (users prefixed `Lab-EntraFund-`, groups prefixed `Grp-EntraFund-`) in the `contoso.onmicrosoft.com` initial domain — never on real staff, production groups, or the synchronized (on-prem) objects.

• **Why required:** Learners practice creating and changing objects; doing that on production identities risks breaking sign-in. A named sandbox keeps every lab reversible and obvious to clean up.

• **How to verify:** Confirm the kickoff states the sandbox rule; confirm the sandbox objects carry the agreed prefixes. `EKIDFUND-Validate.ps1` lists the sandbox objects it finds.

• **How to implement if missing:** State it at kickoff; create the sandbox with `EKIDFUND-LabSetup.ps1` (preview by default); if the tenant is synced from AD, do **not** create lab users in synced OUs — keep them cloud-only.

R4. Break-glass accounts protected (before any Conditional Access lab)

• **Description:** Two cloud-only emergency access (break-glass) accounts exist (global requirement **G4**) and are known to the team, so the Lab 6 **report-only** Conditional Access policy can be built with confidence.

• **Why required:** Even a report-only CA lab is a good moment to teach the anti-lockout discipline — break-glass is excluded from CA policies and never enrolled in a policy that could block it. The lab reinforces the habit before anyone builds a real, enforced policy elsewhere (ENTZTCA).

• **How to verify:** Entra -> **Users**; confirm two cloud-only `@onmicrosoft.com` admin accounts exist. `EKIDFUND-Discovery.ps1` flags whether break-glass-style accounts are present.

• **How to implement if missing:** Create them per **G4** before the Conditional Access lab; store the credentials securely; the lab policy is report-only so it cannot enforce, but the habit is taught regardless.

R5. Entra ID licensing awareness (P1/P2 and add-ons)

• **Description:** Knowledge of what the tenant is licensed for — Microsoft 365 core, **Entra ID P1** (Conditional Access, group-based licensing), **Entra ID P2** (Identity Protection risk, PIM), and any add-ons (Identity Governance, Global Secure Access, Verified ID).

• **Why required:** The licensing/admin-role lab teaches which features are available; some labs (report-only Conditional Access) assume **P1**. *Verify current availability — bundles change and differ by cloud.*

• **How to verify:** M365 admin center -> **Billing -> Licenses**; or `EKIDFUND-Discovery.ps1` reports the detected licensing tier (P1/P2 service plans).

• **How to implement if missing:** If the tenant is core-only (no P1), the Conditional Access lab

becomes a **read-only tour** of the CA blade instead of a report-only policy build — note the gap; every other lab runs in full.

R6. The PowerShell + KQL helper toolchain

- **Description:** A workstation with the **Microsoft Graph PowerShell SDK** installed, signed in with

an MFA-capable account holding at least read scopes; plus access to **sign-in and audit logs** (in the portal, or a Log Analytics workspace where sign-in logs are exported) for the KQL labs.

- **Why required:** The five delivered helpers check readiness, set up the sandbox (gated), run the

sign-in/audit KQL lab pack context, validate skills, and report cohort progress.

- **How to verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version; `Connect-MgGraph`

succeeds; sign-in logs load in **Entra -> Monitoring -> Sign-in logs**.

- **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`; sign in. For

Government cloud, connect with the matching environment/endpoints — *verify the correct endpoints for your cloud*.

Checklist

- Portal access to `entra.microsoft.com` confirmed for every learner (**R1**)
- `Grp-EntraFund-Cohort` created; learners hold read roles; instructor holds setup roles (**R2**)
- Cloud-only lab-sandbox posture agreed; naming prefixes set (**R3**)
- Break-glass accounts confirmed before the Conditional Access lab (**R4**, **G4**)
- Licensing tier known (P1 for the report-only CA lab; note gaps) (**R5**)
- Microsoft Graph SDK installed and consented; sign-in logs reachable (**R6**)

Global Requirements — Entra ID Solutions (category-level)

These requirements apply to every solution in the Entra ID Solutions category. They are published once at the category level so individual listings stay small. Confirm these before deploying any identity solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *The identity plane is high risk. A mistake here breaks sign-in for everyone, so every change is staged and a break-glass account is kept ready before you touch anything.*

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID tenant. Why: It is the cloud identity backbone every other identity service builds on. Verify: Sign in to `https://entra.microsoft.com` as an administrator. Implement: Create a Microsoft 365 or Azure subscription, which provisions a tenant.

G2. Entra ID licensing (P1, P2 where noted)

Description: Microsoft Entra ID P1 (or P2 for risk-based and PIM features). Why: Hybrid features such as seamless sign-in, password writeback, and Conditional Access require P1+. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2 (often bundled in M365 E3/E5 or EMS). Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

G3. Administrative roles

Description: Hybrid Identity Administrator and Global Administrator for the operator. Why: Configuring directory synchronization and authentication methods needs these elevated roles. Verify: Entra -> `Roles and administrators`; confirm `Hybrid Identity Administrator` (and a Global Administrator for the one-time first-sync consent). Implement: Assign `Hybrid Identity Administrator`; reserve `Global Administrator` for tenant-wide and first-sync changes.

G4. Break-glass (emergency access) accounts

Description: At least two cloud-only emergency access accounts, excluded from Conditional Access and MFA-broad policies. Why: If sync, federation, or an auth-method change locks out admins, break-glass accounts get you back in. Verify: Entra -> `Users`; confirm two cloud-only accounts (`@onmicrosoft.com`) with strong stored credentials and documented use. Implement: Create two cloud-only Global Administrator accounts; exclude them from policies; store credentials securely and monitor sign-ins.

G5. Healthy on-premises Active Directory

Description: A functional AD forest/domain with healthy replication and DNS. Why: Synchronization reads from AD; replication or DNS faults corrupt or stall the sync. Verify: Run `dcdiag` and `repadmin /replsummary` on a domain controller with no errors; run `HYBID-Discovery.ps1`. Implement: Resolve AD replication, DNS, and time-sync issues before synchronizing.

G6. An Entra Connect host server

Description: A domain-joined Windows Server (2016+), patched, for Microsoft Entra Connect Sync (or a lightweight host with the Cloud Sync agent). Why: Connect Sync runs as a service on this host and must reach both AD and Entra ID. Verify: A supported, domain-joined server with TLS 1.2, .NET, and PowerShell available; run `HYBID-Discovery.ps1`. Implement: Provision a dedicated server (not a domain controller for production), patch it, and enable TLS 1.2.

G7. UPN / domain hygiene

Description: User principal names use a public, verified, routable domain (not `.local`). Why: A non-routable UPN suffix produces sign-in problems and `onmicrosoft.com` fallbacks in the cloud. Verify: Entra -> `Identity -> Settings -> Domain names` shows the public domain Verified; AD UPN suffixes match it; run `HYBID-Discovery.ps1`. Implement: Add and verify the public domain in Entra; add the matching UPN suffix in AD and update user UPNs.

G8. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS (443) from the Connect host to Entra ID, plus required Microsoft 365 service URLs. Why: Synchronization, authentication-agent traffic, and tokens depend on these endpoints. Verify: From the Connect host, reach `https://login.microsoftonline.com` and the Microsoft 365 published URL/IP ranges. Implement: Allow the Microsoft 365 / Entra service URLs and IP ranges through the proxy/firewall.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Self-Deploy: Microsoft Entra Identity Fundamentals

This is the exact, ordered guide — a sequence of **guided hands-on labs**. Every object you create is **named** with a consistent convention — `<Purpose> - <Scope> - <Stage>` — and the same name is reused in later labs, the knowledge-check, and the scripts. Labs run **read-only first**, and every object you create is a **cloud-only sandbox** object you can delete afterward. The only Conditional Access object built is created in **report-only** and can never enforce.

> ***Before you start:*** *confirm portal access (**R1**), the cohort and least-privilege roles (**R2**), the cloud-only sandbox posture (**R3**), break-glass accounts (**R4**, **G4**), your licensing tier (**R5**), and the toolchain (**R6**). This guide **explores and practices** on a sandbox; it does not reconfigure production identity.*

Object naming convention (used everywhere below and in the scripts): `<Purpose> - <Scope> - <Stage>`**.** Create these once and reuse the exact names in testing, troubleshooting, and scripts:

Object	Name	What it is
Learner cohort group	`Grp-EntraFund-Cohort`	Entra security group of kit learners
Lab practice team group	`Grp-EntraFund-LabTeam-Pilot`	Assigned security group used in the groups & CA labs
Lab dynamic group	`Grp-EntraFund-DynamicDept-Pilot`	Dynamic membership group (rule-driven) demo
Lab cloud-only test user	`Lab-EntraFund-User1`	UPN `lab-entrafund-user1@contoso.onmicrosoft.com` — a disposable test identity
Lab app registration	`App-EntraFund-LabApp-Pilot`	An app registration created to see the registration vs enterprise-app split
Lab Conditional Access policy	`CAFund-RequireMFA-LabTeam-ReportOnly`	A **report-only** CA policy scoped to the lab team only
Knowledge-check	`Check-EntraFund-Pilot`	The scored end-of-kit knowledge-check

The runnable sign-in/audit queries for the sign-in and MFA labs live in the delivered `EKIDFUND-Labs.kql` pack, organized by lab. You can paste them into **Entra -> Monitoring -> Sign-in logs** log-analytics view (where exported) or run the read-only equivalents from PowerShell.

Phase 0 — Assess (read-only, changes nothing)

Step 0.1 — Confirm readiness

1. Sign in to the **Microsoft Entra admin center** -> `https://entra.microsoft.com` and confirm the

Overview blade shows your tenant name and tenant ID.

1. From PowerShell, take a read-only picture of the tenant fundamentals:

```
pwsh -File .\EKIDFUND-Discovery.ps1
```

It reports, read-only: user and guest counts, group counts (security vs M365, assigned vs dynamic), directory-role assignments (and whether Global Administrator is over-assigned), app registration and enterprise-app counts, the licensing tier (P1/P2), MFA-registration coverage, and any existing Conditional Access policies. **Nothing is modified.** Save the output — it is your before picture and a teaching prop.

Step 0.2 — Create the learner cohort

1. In the Entra admin center -> **Groups -> All groups -> New group**.
2. Type **Security**; **Group name** `Grp-EntraFund-Cohort`; add the learners as members.
3. Confirm each learner holds a read role (**Global Reader** / **Directory Readers**) and the

instructor holds the setup roles (**R2**).

Phase 1 — Configure (stand up the cloud-only lab sandbox)

Step 1.1 — Create the lab sandbox (instructor)

1. Preview what the sandbox setup would create (changes nothing):

```
pwsh -File .\EKIDFUND-LabSetup.ps1
```

1. Review the preview, then create the sandbox objects (still small, cloud-only, disposable):

```
pwsh -File .\EKIDFUND-LabSetup.ps1 -WhatIf:$false
```

It creates `Grp-EntraFund-LabTeam-Pilot`, `Grp-EntraFund-DynamicDept-Pilot`, the cloud-only test user `Lab-EntraFund-User1`, and the app registration `App-EntraFund-LabApp-Pilot`. All are named for easy cleanup. *You can also create each object by hand in the labs below — the script just makes setup and teardown one command.*

Phase 2 — Pilot (the guided labs)

Run each lab with the cohort. Each lab states the **concept**, the **task**, and the **check**.

Lab 1 — Tenant tour: what you are looking at

• **Concept.** A tenant is your isolated directory instance. It has an initial domain

(`contoso.onmicrosoft.com`), a tenant ID, verified custom domains, and every identity/config below.

• **Task.** In **Overview**, find the tenant name, tenant ID, and primary domain. Open **Identity ->

Settings -> Domain names **and note which domains are** Verified**. Open the left-nav and locate Users, Groups, Roles and administrators, Applications, and Protection.

• **Check.** You can state your tenant ID, primary domain, and where each major blade lives.

Lab 2 — Users and identities

• **Concept.** Entra holds **member** users, **guest** users (B2B), and **service principals**

(apps/automation, not people). Each has a UPN, object ID, and attributes.

• **Task.** Open **Users -> All users**; open `Lab-EntraFund-User1` and read its UPN, object ID,

usage location, and assigned roles/groups. Filter the list to **Guest** user type and note how a guest UPN differs (#EXT#). *Do not modify a real user.*

• **Check.** You can tell a member from a guest from a service principal, and read a user's UPN and

object ID.

Lab 3 — Groups and dynamic membership

• **Concept.** A **security group** targets access/policy to many users; a **Microsoft 365 group**

also backs a mailbox/Team. Membership is **assigned** or **dynamic** (rule-driven).

• **Task.** Open `Grp-EntraFund-LabTeam-Pilot` and add `Lab-EntraFund-User1` as a member (assigned).

Open `Grp-EntraFund-DynamicDept-Pilot` and read its **dynamic membership rule** (e.g. `user.department -eq "LabDept"`); set `Lab-EntraFund-User1`'s department to `LabDept` and watch it join automatically. *These are sandbox groups only.*

• **Check.** You can create/target a security group and explain assigned vs dynamic membership.

Lab 4 — Roles and RBAC (least privilege)

• **Concept.** Built-in **directory roles** bundle permissions. Global Administrator is

all-powerful and should be rare; most tasks need a narrow role (User Administrator, Helpdesk Administrator, Security Reader).

• **Task.** Open **Roles and administrators**; open **Global Administrator** and read its members

(note how few there should be). Open **Helpdesk Administrator** and read what it can and cannot do. Using `EKIDFUND-Discovery.ps1` output, discuss whether any role is over-assigned. *Read-only — do not change a real assignment.*

• **Check.** You can name three least-privilege roles, explain why Global Administrator is guarded,

and read who holds a role.

Lab 5 — App registrations vs enterprise applications

• **Concept.** An **app registration** is the *definition* of an app (identity, redirect URIs, API

permissions, secrets). An **enterprise application** (service principal) is the *instance* users sign in to and that you assign/consent/govern.

• **Task.** Open **App registrations** and open `App-EntraFund-LabApp-Pilot`; read its Application (client) ID, redirect URIs, and **API permissions** (note delegated vs application permissions, and what "admin consent" means — do not grant consent in the lab). Then open **Enterprise applications** and find the matching service principal — the *instance* of the same app. *Read-only.*

• **Check.** You can explain the registration vs enterprise-app split and where API permissions and consent live.

Lab 6 — Sign-in, MFA, and a report-only Conditional Access policy

• **Concept.** A sign-in is authenticated, then evaluated by **Conditional Access** (an if-then engine) before a token is issued. **MFA** adds a second factor; **phishing-resistant** methods (FIDO2/passkey) are the strongest. **Report-only** lets a policy be fully evaluated and logged **without enforcing** — the safe way to learn CA.

• **Task (read the sign-in logs).** Open **Monitoring -> Sign-in logs**; open one sign-in and read its **Conditional Access** tab, **Authentication details** (which method satisfied MFA), and status. Run the sign-in KQL from `EKIDFUND-Labs.kql` (Lab 6 section) to summarize sign-ins by method and result.

• **Task (build a report-only policy — P1 required, safe).** In **Protection -> Conditional Access -> Policies -> New policy**, create** `CAFund-RequireMFA-LabTeam-ReportOnly`**:** Assignments -> Users -> Include `Grp-EntraFund-LabTeam-Pilot`**;** Exclude **your break-glass accounts (**R4/G4**);** Target resources -> All cloud apps**;** Grant -> Require multifactor authentication**; set** Enable policy = Report-only**. Save. *It is report-only — it evaluates and logs but cannot block anyone.* If the tenant has no P1, make this a** read-only tour **of the CA blade instead (**R5**).

• **Check.** You can trace a sign-in through MFA and Conditional Access in the logs, and you built a policy that is scoped to the lab team, excludes break-glass, and is **report-only**.

Phase 3 — Validate (capstone + knowledge-check)

1. **Capstone.** Each learner, unaided, does the following on the sandbox: create a new cloud-only test user, add it to `Grp-EntraFund-LabTeam-Pilot`, confirm the report-only CA policy *would* apply to it (via **Conditional Access -> What If**), and explain each step aloud. *No production object is touched; the CA policy stays report-only.*

1. **Knowledge-check.** Score the knowledge-check `Check-EntraFund-Pilot`:

```
pwsh -File .\EKIDFUND-Validate.ps1
```

It walks the pass/fail gates in `05-testing.md` — sandbox objects exist, the CA lab policy is report-only and excludes break-glass, licensing is as expected — and prints the knowledge-check answer key for the instructor to score.

1. **Cohort report.** Summarize the fundamentals picture and cohort progress:

```
pwsh -File .\EKIDFUND-Report.ps1 -OutputCsv .\ekidfund-cohort.csv
```

Phase 4 — Expand (operate with confidence)

1. **Keep or tear down the sandbox.** If the kit was a one-off, remove the sandbox objects (Step "Rollback" below). If you will re-run for new hires, keep them and re-use the runbook.

1. **Progress to the advanced solutions.** With the fundamentals in place, the natural next steps are the **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** (design a real, enforced baseline), **Privileged Identity Management (ENTPIM)** (just-in-time admin), **Passwordless & Passkey (ENTPASSKEY)** (phishing-resistant MFA), and **Identity Governance & Access Reviews (ENTIGA)**.

1. **Adopt the runbook.** Move the take-home fundamentals runbook and glossary into the team's standing operations documentation so the grounding is reused, not re-explained.

Rollback / safe-off

- **Exploration labs (1, 2, 4, 5):** read-only — nothing to roll back.
- **Sandbox objects (Labs 1–5):** delete `Grp-EntraFund-LabTeam-Pilot`, `Grp-EntraFund-DynamicDept-Pilot`, `Lab-EntraFund-User1`, and `App-EntraFund-LabApp-Pilot` (and its enterprise-app) if the kit was one-off — or run `EKIDFUND-LabSetup.ps1 -Remove -WhatIf:$false` after previewing.
- **Conditional Access lab policy:** the policy is **report-only** and cannot enforce; set it to `Off` or delete `CAFund-RequireMFA-LabTeam-ReportOnly` to remove it entirely. Break-glass is excluded throughout, so there is no lock-out risk.

5. Testing & Validation — Self-Deploy: Microsoft Entra Identity Fundamentals

Validate the kit against explicit **pass/fail gates** — both that the **lab sandbox** is set up correctly and that the **skills** landed. Run the scripts and walk the manual checks. The automated gates are produced by `EKIDFUND-Validate.ps1`; readiness comes from `EKIDFUND-Discovery.ps1`, the sign-in labs from `EKIDFUND-Labs.kql`, and cohort evidence from `EKIDFUND-Report.ps1`.

Each gate lists **What good looks like / How to test / If it fails**.

Gate 1 — The tenant is reachable and the fundamentals picture is captured

- **What good looks like:** The Entra admin center loads; `EKIDFUND-Discovery.ps1` returns the tenant fundamentals (users, groups, roles, apps, licensing).
- **How to test:** Run `EKIDFUND-Discovery.ps1`; confirm it prints user/group/role/app counts, the detected P1/P2 tier, and any existing CA policies without error.
- **If it fails:** Confirm portal access (**R1**), the Graph SDK is installed and consented (**R6**),

and the account holds at least a read role (**R2**).

Gate 2 — The learner cohort exists

• **What good looks like:** `Grp-EntraFund-Cohort` exists with the learners as members; learners hold read roles.

• **How to test:** Entra -> **Groups**; confirm the cohort group and membership. `EKIDFUND-Discovery.ps1` lists the cohort group where present.

• **If it fails:** Create the cohort (Step 0.2) and assign read roles (**R2**).

Gate 3 — The cloud-only lab sandbox exists and is correctly named

• **What good looks like:** `Grp-EntraFund-LabTeam-Pilot`, `Grp-EntraFund-DynamicDept-Pilot`, `Lab-EntraFund-User1`, and `App-EntraFund-LabApp-Pilot` all exist, are **cloud-only**, and carry the agreed prefixes.

• **How to test:** Run `EKIDFUND-Validate.ps1`; confirm it lists each sandbox object as present and cloud-only. Or check them by name in the portal.

• **If it fails:** Create them with `EKIDFUND-LabSetup.ps1 -WhatIf:$false`, or by hand in Labs 3 and 5.

If any is synchronized from on-prem, recreate it cloud-only (**R3**).

Gate 4 — Learners can read a user, group, role, and app unaided

• **What good looks like:** In the capstone, learners open a user and read its UPN/object ID, target a group, name a least-privilege role, and explain the registration vs enterprise-app split — without help.

• **How to test:** Observe the capstone; confirm the learner performs Labs 2–5 actions on the sandbox unaided and explains each.

• **If it fails:** Re-run the specific lab (Labs 2–5) the learner struggled with before repeating the capstone.

Gate 5 — The sign-in labs run and the CA lab policy is report-only

• **What good looks like:** The learner can open a sign-in and read its Conditional Access and authentication-method details; `CAFund-RequireMFA-LabTeam-ReportOnly` exists in **report-only**, scoped to `Grp-EntraFund-LabTeam-Pilot`, with break-glass excluded.

• **How to test:** Review a sign-in in **Monitoring -> Sign-in logs**; run a `EKIDFUND-Labs.kql` query; run `EKIDFUND-Validate.ps1`, which confirms the CA policy state is report-only and lists its scope and exclusions. *If the tenant has no P1, mark the policy build N/A and confirm the CA blade tour

instead.*

• **If it fails:** Set the policy state back to **Report-only** if it drifted; add the break-glass exclusion if missing; re-scope to the lab team only (Lab 6).

Gate 6 — Break-glass is excluded from the CA lab policy

• **What good looks like:** The two break-glass accounts are **excluded** from `CAFund-RequireMFA-LabTeam-ReportOnly` — teaching the anti-lockout habit even in report-only.

• **How to test:** `EKIDFUND-Validate.ps1` reports the policy's user exclusions; confirm break-glass is listed. Or check **Assignments -> Users -> Exclude** in the portal.

• **If it fails:** Add the break-glass accounts (or their exclusion group) to the policy's exclusions (**R4/G4**).

Gate 7 — The knowledge-check passes

• **What good looks like:** Each learner passes `Check-EntraFund-Pilot` (tenant, users vs guests vs service principals, groups & membership, least-privilege roles, registration vs enterprise app, sign-in/MFA/CA basics, licensing).

• **How to test:** Run `EKIDFUND-Validate.ps1`; it prints the knowledge-check answer key mapped to the lab that teaches each item, for the instructor to score.

• **If it fails:** Revisit the mapped lab for any question a learner missed; the knowledge-check tests **concepts**, not tenant-specific values.

Gate 8 — Read-only-first posture held; roles least-privilege

• **What good looks like:** No production identity was changed during the kit; learners held read roles and only the instructor created sandbox objects; the CA lab never left report-only.

• **How to test:** Review **Roles and administrators** (R2) and the audit log; confirm learner activity was read/sandbox-only.

• **If it fails:** Tighten role assignments; reaffirm the sandbox posture (**R3**) and re-run affected labs on sandbox objects only.

Validation summary

Gate	Check	Pass condition
1	Tenant reachable + fundamentals captured	`EKIDFUND-Discovery.ps1` returns the picture

Gate	Check	Pass condition
2	Cohort exists	`Grp-EntraFund-Cohort` present with members
3	Lab sandbox exists, cloud-only	All four sandbox objects present and correctly named
4	Read user/group/role/app unaided	Capstone Labs 2–5 performed without help
5	Sign-in labs + CA lab report-only	Sign-in read; `CAFund-RequireMFA-LabTeam-ReportOnly` is report-only
6	Break-glass excluded	Break-glass excluded from the CA lab policy
7	Knowledge-check passes	`Check-EntraFund-Pilot` scored to threshold
8	Read-only-first + least-privilege	No production change; roles scoped

All gates pass ⇒ the cohort is grounded; proceed to **Expand** (operate with confidence; progress to the advanced solutions). Any fail ⇒ reinforce the mapped lab and re-check.

6. Troubleshooting — Self-Deploy: Microsoft Entra Identity Fundamentals

The common failure modes, each as **symptom** → **cause** → **fix**. Most kit issues are a missing role, a sandbox object created the wrong way, dynamic-membership lag, or a licensing surprise — not a broken tenant. Use `EKIDFUND-Discovery.ps1` for the read-only picture and `EKIDFUND-Validate.ps1` for the gate checks.

1. A learner cannot open a blade / everything is greyed out

- **Symptom:** The Users, Groups, or Roles blade is missing or read-only actions are unavailable.
- **Cause:** The learner has no directory role, or too low a one.
- **Fix:** Assign at least **Global Reader** (or **Directory Readers**) so learners can explore read-only (**R2**). Only the instructor needs create/change roles. Confirm portal reachability (**R1/G8**).

2. `Connect-MgGraph` fails or a script returns 403

- **Symptom:** A helper cannot connect, or a call is forbidden.
- **Cause:** The Graph SDK is not installed, scopes were not consented, the wrong cloud endpoint was used, or the account lacks the role.

• **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`; re-run and consent the read scopes in

the script header; for Government cloud pass the matching `-Environment`; confirm the account holds a read role (**R6/R2**). *Verify the correct endpoints for your cloud.*

3. The lab sandbox setup won't create objects

• **Symptom:** `EKIDFUND-LabSetup.ps1 -WhatIf:$false` fails to create a group, user, or app.

• **Cause:** The account lacks the create role (**User Administrator** / **Groups Administrator** / **Application Administrator**), or the initial domain name is different in your tenant.

• **Fix:** Run the setup as the **instructor** with the create roles; if your initial domain is not `contoso.onmicrosoft.com`, pass your real `*.onmicrosoft.com` domain via the script's `-Domain` parameter so the test UPN is valid.

4. A lab test user cannot be edited (it's synced)

• **Symptom:** Fields on a user are read-only in Entra ("managed on-premises").

• **Cause:** The object is **synchronized from Active Directory**, not cloud-only — synced attributes are managed on-prem.

• **Fix:** Do labs on **cloud-only** sandbox users (`Lab-EntraFund-User1`), never on synced accounts (**R3**). Create a fresh cloud-only user if needed.

5. Dynamic-group membership didn't update

• **Symptom:** After setting `Lab-EntraFund-User1`'s department to `LabDept`, it isn't in `Grp-EntraFund-DynamicDept-Pilot` yet.

• **Cause:** Dynamic membership is evaluated **asynchronously** — it can take minutes (longer in large tenants), and the rule must match exactly.

• **Fix:** Wait and refresh; confirm the rule (`user.department -eq "LabDept"`) matches the attribute value exactly (case and spelling); check the group's **Membership processing status**.

6. The Conditional Access lab policy isn't report-only (or can't be built)

• **Symptom:** `CAFund-RequireMFA-LabTeam-ReportOnly` shows as On/Off instead of Report-only, or the

CA blade is unavailable.

• **Cause:** The state was changed, or the tenant has no **Entra ID P1** (Conditional Access is a P1 feature).

• **Fix:** Set **Enable policy = Report-only** (Lab 6); it must never be On for this kit. If there is

no P1, make Lab 6 a **read-only tour** of the CA blade instead of a policy build, and note the gap (**R5**). *Verify licensing.*

7. Break-glass is not excluded from the CA lab policy

- **Symptom:** `EKIDFUND-Validate.ps1` flags the CA lab policy as missing the break-glass exclusion.
- **Cause:** The exclusion was not added when the policy was created.
- **Fix:** Add the two break-glass accounts (or their exclusion group) under **Assignments -> Users -> Exclude **(**R4/G4**). Teaching this habit is a core point of the lab even though the policy is report-only.

8. Sign-in logs show nothing / the KQL lab returns no rows

- **Symptom:** A `EKIDFUND-Labs.kql` query returns 0 rows, or Sign-in logs look empty.
- **Cause:** The time window is too tight, the sandbox user hasn't signed in yet, or (for the Log-Analytics variant) sign-in logs aren't being exported to a workspace.
- **Fix:** Widen the window (`ago(7d)`); have `Lab-EntraFund-User1` sign in once to generate a sign-in; use the **Entra -> Monitoring -> Sign-in logs** portal view if no Log Analytics export exists. *Sign-in-log retention and export are P1/P2 and workspace-dependent — verify in your tenant.*

9. "Which license do I have?" confusion

- **Symptom:** A feature the kit mentions (risk policies, PIM, Governance) isn't visible.
- **Cause:** Those need **Entra ID P2** or an add-on the tenant may not have.
- **Fix:** Run `EKIDFUND-Discovery.ps1` (licensing section) or check **Billing -> Licenses**; the kit flags P2/add-on features as *verify availability*. Everything in the core labs runs on P1; note any gap and move on.

10. App registration vs enterprise application confusion

- **Symptom:** A learner can't find the app in the expected blade, or edits the wrong object.
- **Cause:** The **registration** (definition, under **App registrations**) and the **enterprise application **(**instance/service principal, under** Enterprise applications**) are two different blades for the same app.
- **Fix:** Re-do Lab 5: open `App-EntraFund-LabApp-Pilot` under **App registrations** for its identity/permissions, then find the matching service principal under **Enterprise applications** for assignment/consent. They are linked but distinct.

7. Best Practices — Self-Deploy: Microsoft Entra Identity Fundamentals

Keep-it-clean guidance for turning a fundamentals kit into a durable, safe operating habit — and where to go next once the grounding is in place.

Learn on a sandbox, never on production

- **Practice on cloud-only, clearly named test objects.** Prefix every lab object (`Lab-EntraFund-`, `Grp-EntraFund-`) and keep them in the `contoso.onmicrosoft.com` initial domain. A new admin should never learn "what does this button do" on a real user's account.
- **Read before you write.** Most identity mistakes are one irreversible change made without understanding. Open, read, and explain an object first (Labs 2, 4, 5 are read-only by design), then change only the sandbox copy.
- **Clean up when you are done.** Delete sandbox objects (or keep a labelled set for re-runs). Stale test users and half-configured apps are their own small risk.

Treat the identity plane as high-risk

- **Guard Global Administrator.** It is all-powerful — keep the count tiny, prefer least-privilege roles (User Administrator, Helpdesk Administrator, Security Reader) for day-to-day work, and make elevation deliberate (PIM, where licensed).
- **Keep break-glass sacred.** Two cloud-only emergency accounts, excluded from Conditional Access, credentials stored securely. Teach the habit in the report-only CA lab so it is second nature before anyone builds an enforced policy.
- **Report-only before enforce, always.** The one CA policy this kit builds is **report-only** — it evaluates and logs but cannot block. Carry that discipline into every real policy you build later: report-only, validate, then enforce in waves.

Build the right mental model

- **Tenant -> identities -> groups -> roles.** Hold the hierarchy in your head: the tenant is the boundary, identities are who signs in, groups are how you manage them in bulk, roles are who can administer what. Almost every task maps onto that model.
- **Registration is the definition; enterprise app is the instance.** The single most common fundamentals gap. An app registration defines an app; the enterprise application (service principal) is what users sign in to and what you assign and govern.
- **Target groups, not individuals.** Assign licenses, app access, and Conditional Access to security groups so the design is legible and scalable — never a pile of per-user assignments.

Verify, don't assume

- **Verify licensing before you promise a feature.** Conditional Access needs **P1**; risk policies

and PIM need **P2**; Governance, Global Secure Access, and Verified ID are further add-ons. Check **Billing -> Licenses** (or `EKIDFUND-Discovery.ps1`) — *bundles change and differ by cloud (Commercial vs GCC / GCC High)*.

• **Verify portal navigation and preview status.** Microsoft renames blades and ships features in preview. Treat the click-paths in this kit as current-at-writing and confirm against your tenant.

Run it as a program, not a one-off class

• **Re-run for new hires.** The take-home runbook, glossary, and knowledge-check make onboarding a new admin repeatable — the sandbox and scripts are reusable.

• **Keep the runbook current.** As your tenant evolves (new domains, new roles, new apps), update the take-home runbook so it stays your team's fundamentals reference.

Where to go next

With the fundamentals in place, the natural next steps in this catalog are:

• **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** — design and roll out a real, enforced CA baseline (this kit built only a report-only teaching policy).

• **Self-Deploy — MFA & Conditional Access (EKMFACA)** — go deeper on MFA methods, CA design patterns/personas, and safe report-only/What-If testing as a team-enablement follow-on.

• **Privileged Identity Management (ENTPIM)** — just-in-time, approval-gated admin elevation.

• **Passwordless & Passkey (ENTPASSKEY)** — deploy phishing-resistant MFA.

• **Identity Governance & Access Reviews (ENTIGA)** — periodic access certification and entitlement management.

• **Hybrid Identity** and **Enterprise-App & Service-Principal Governance** — extend the grounding to AD-to-Entra sync and the app estate.

8. Deliverables Summary — Self-Deploy: Microsoft Entra Identity Fundamentals

Everything included in this kit, and what each piece is for.

Documentation

Document	Purpose
Executive Summary (`executive-summary.md`)	One-page framing: the "inherited a tenant, nobody knows how it works" problem, the enablement solution (concepts + guided labs + take-home runbook/glossary + knowledge-check), and the boundary (skills product; adjacent to the implementation solutions; practices on a cloud-only sandbox, report-only for CA).
1. Solution Overview (`01-solution-overview.md`)	What it is, how Entra works (tenant, identities, groups, roles/RBAC, app registrations vs enterprise apps, sign-in/MFA/CA, licensing), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used.
2. Requirements Guide (`02-requirements.md`)	Solution-specific prerequisites (portal access, cohort + least-privilege roles, cloud-only sandbox posture, break-glass, licensing awareness, toolchain) — layered on the shared `global-requirements-entra-id-solutions.md` (G1–G8).
Global Requirements (`global-requirements-entra-id-solutions.md`)	Category-wide prerequisites, published once (tenant, licensing, admin roles, break-glass, network access).
3. Implementation Guide (`03-implementation.md`)	The guided labs: assess readiness, create the cohort and lab sandbox, run Labs 1–6 (tenant tour → users → groups → roles/RBAC → app registrations → sign-in/MFA/report-only CA), then capstone + knowledge-check, then operate with confidence — every object named, with rollback.
5. Testing & Validation (`05-testing.md`)	Eight pass/fail gates covering both sandbox setup and skills landed (read a user/group/role/app unaided, sign-in labs, CA lab report-only, break-glass excluded, knowledge-check passes), referencing the scripts by name.
6. Troubleshooting (`06-troubleshooting.md`)	The common failure modes (portal/permission errors, sandbox object issues, dynamic-membership lag, CA lab not report-only, sign-in-log gaps, licensing surprises) as symptom → cause → fix.
7. Best Practices (`07-best-practices.md`)	Learn-on-a-sandbox, treat-identity-as-high-risk, build-the-right-mental-model, verify-don't-assume, run-it-as-a-program — plus where to go next in the catalog.

Lab helpers (5)

All helpers authenticate **interactively** (no credentials in code) and use the **Microsoft Graph PowerShell SDK**; the KQL pack runs over Entra sign-in/audit logs. The discovery, validation, and report helpers are **read-only**; the sandbox setup is **preview (`-WhatIf`) by default** and gated. Government-cloud tenants pass the matching environment — *verify endpoints for your cloud*.

File	Type	What it does
`EKIDFUND-Discovery.ps1`	Read-only	Inventories the tenant fundamentals: user/guest counts, group counts (security vs M365, assigned vs dynamic), directory-role assignments (flags Global-Admin over-assignment), app registration / enterprise-app counts, licensing tier (P1/P2), MFA-registration coverage, and existing CA policies. Changes nothing.
`EKIDFUND-LabSetup.ps1`	Gated write (preview by default)	Creates the cloud-only lab sandbox — `Grp-EntraFund-LabTeam-Pilot`, `Grp-EntraFund-DynamicDept-Pilot`, `Lab-EntraFund-User1`, `App-EntraFund-LabApp-Pilot`. Defaults to `-WhatIf` (preview); `-Remove` tears the sandbox down. Never touches production objects.
`EKIDFUND-Labs.kql`	KQL lab pack	The runnable sign-in/audit query pack for the sign-in and MFA labs — sign-ins by method and result, MFA-registration coverage, Conditional Access "would-apply" results, and audit trail of the sandbox changes. Read-only; the take-home query starter.
`EKIDFUND-Validate.ps1`	Read-only	Runs the `05-testing.md` gates as PASS/FAIL/MANUAL lines: sandbox objects present and cloud-only, the CA lab policy is report-only with break-glass excluded, licensing as expected — and prints the knowledge-check `Check-EntraFund-Pilot` answer key for scoring.
`EKIDFUND-Report.ps1`	Read-only	Reports the fundamentals picture and cohort progress (objects created, CA lab state, knowledge-check results) and exports to CSV.

Named objects (verbatim — used across docs, scripts, and tests)

- **Learner cohort group:** `Grp-EntraFund-Cohort`
- **Lab practice team group:** `Grp-EntraFund-LabTeam-Pilot`
- **Lab dynamic group:** `Grp-EntraFund-DynamicDept-Pilot`
- **Lab cloud-only test user:** `Lab-EntraFund-User1`
- **Lab app registration:** `App-EntraFund-LabApp-Pilot`
- **Lab Conditional Access policy:** `CAFund-RequireMFA-LabTeam-ReportOnly` (report-only)
- **Knowledge-check:** `Check-EntraFund-Pilot`

What this kit deliberately does NOT do

- It does **not** design or roll out a production Conditional Access baseline — that is **ENTZTCA** and **Conditional Access Integration**. This kit builds one **report-only** teaching policy.
- It does **not** deploy PIM, passwordless, governance, or hybrid sync — those are **ENTPIM**, **ENTPASSKEY**, **ENTIGA**, and **Hybrid Identity**. It introduces the concepts and cross-references.
- It does **not** change production identity — labs run on a **cloud-only sandbox** and any CA object is **report-only**.
- It does **not** decide business/security policy — which roles are "privileged," which MFA methods are mandated, which apps are sanctioned are the customer's determinations; the kit provides the skill and the labs.
- It does **not** invent licensing or preview facts — where availability may differ by tenant/cloud (P1/P2, Governance, Global Secure Access, Verified ID, preview features) it says **verify in your tenant**.

How it is delivered

This kit is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the KQL lab pack and all five lab helpers, in a single ZIP (`EKIDFUND-Complete-Solution.zip`) — yours to keep and re-run for every new admin.

Outcome

A team grounded in Microsoft Entra fundamentals — able to read the directory, manage users and groups, apply least-privilege roles, recognize app registrations, and trace a sign-in through MFA and Conditional Access — ready to operate the tenant safely and to progress to the advanced identity solutions.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EKIDFUND-Discovery.ps1

Purpose: A read-only "before picture" of the tenant fundamentals, used to ground the enablement kit: user and guest counts, group counts (security vs Microsoft 365, assigned vs dynamic), directory-role assignments (flagging Global Administrator over-assignment), app-registration and enterprise-application counts, the detected licensing tier (Entra ID P1 / P2), MFA registration coverage, and any existing Conditional Access policies. This is a teaching prop and a readiness check — it changes NOTHING in the tenant. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once pwsh -File .\EKIDFUND-Discovery.ps1 pwsh -File .\EKIDFUND-Discovery.ps1 -CsvPath .\ekidfund-discovery.csv Consents to read-only scopes (Directory.Read.All, Policy.Read.All, RoleManagement.Read.Directory, AuditLog.Read.All, Application.Read.All, UserAuthenticationMethod.Read.All). No changes. Government cloud: pass -Environment USGov (or USGovDoD). EXPECTED RESULTS A console summary of the fundamentals with a few "teach this" callouts (e.g. Global Admin count, licensing tier, MFA coverage). Nothing is modified. Optional CSV export of the summary rows. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once pwsh -File .\EKIDFUND-Discovery.ps1 pwsh -File .\EKIDFUND-Discovery.ps1 -CsvPath .\ekidfund-discovery.csv Consents to read-only scopes (Directory.Read.All, Policy.Read.All, RoleManagement.Read.Directory, AuditLog.Read.All, Application.Read.All, UserAuthenticationMethod.Read.All). No changes. Government cloud: pass -Environment USGov (or USGovDoD). EXPECTED RESULTS A console summary of the fundamentals with a few "teach this" callouts (e.g. Global Admin count, licensing tier, MFA coverage). Nothing is modified. Optional CSV export of the summary rows. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKIDFUND-LabSetup.ps1

Purpose: Stands up a small, clearly-named, CLOUD-ONLY lab sandbox so learners practice on disposable test objects instead of production identities: the practice team group Grp-EntraFund-LabTeam-Pilot, the dynamic group Grp-EntraFund-DynamicDept-Pilot (rule user.department -eq "LabDept"), the cloud-only test user Lab-EntraFund-User1, and the app registration App-EntraFund-LabApp-Pilot. Every object is named for one-command teardown. This is the ONLY write script in the kit — it defaults to -WhatIf (preview) and changes nothing until you re-run with -WhatIf:$false. -Remove tears the sandbox down (also preview by default). It never touches any object it did not create by these exact names. HOW TO RUN Preview what would be created (default, no changes): pwsh -File .\EKIDFUND-LabSetup.ps1 Create the sandbox after review: pwsh -File .\EKIDFUND-LabSetup.ps1 -WhatIf:$false Use your real initial domain (default contoso.onmicrosoft.com placeholder): pwsh -File .\EKIDFUND-LabSetup.ps1 -Domain yourtenant.onmicrosoft.com -WhatIf:$false Tear the sandbox down (preview first, then apply): pwsh -File .\EKIDFUND-LabSetup.ps1 -Remove pwsh -File .\EKIDFUND-LabSetup.ps1 -Remove -WhatIf:$false Consents to Group.ReadWrite.All, User.ReadWrite.All, Application.ReadWrite.All, Directory.Read.All. Government cloud: pass -Environment USGov (or USGovDoD). EXPECTED RESULTS In preview: the sandbox objects that WOULD be created

(or removed) and nothing else. When applied: the four cloud-only sandbox objects exist (or are gone). Fully reversible via -Remove. .NOTES White-label — Entra ID Solutions. Write action — preview (-WhatIf) by default. Cloud-only sandbox objects ONLY; never touches production identities. No credentials in code. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$Domain = 'contoso.onmicrosoft.com', [switch]$Remove, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: Preview what would be created (default, no changes): pwsh -File .\EKIDFUND-LabSetup.ps1 Create the sandbox after review: pwsh -File .\EKIDFUND-LabSetup.ps1 -WhatIf:$false Use your real initial domain (default contoso.onmicrosoft.com placeholder): pwsh -File .\EKIDFUND-LabSetup.ps1 -Domain yourtenant.onmicrosoft.com -WhatIf:$false Tear the sandbox down (preview first, then apply): pwsh -File .\EKIDFUND-LabSetup.ps1 -Remove pwsh -File .\EKIDFUND-LabSetup.ps1 -Remove -WhatIf:$false Consents to Group.ReadWrite.All, User.ReadWrite.All, Application.ReadWrite.All, Directory.Read.All. Government cloud: pass -Environment USGov (or USGovDoD). EXPECTED RESULTS In preview: the sandbox objects that WOULD be created (or removed) and nothing else. When applied: the four cloud-only sandbox objects exist (or are gone). Fully reversible via -Remove. .NOTES White-label — Entra ID Solutions. Write action — preview (-WhatIf) by default. Cloud-only sandbox objects ONLY; never touches production identities. No credentials in code. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$Domain = 'contoso.onmicrosoft.com', [switch]$Remove, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKIDFUND-Labs.kql

Purpose: // The runnable KQL for the sign-in / MFA / Conditional Access labs (Lab 6) and the // audit-trail checks. One labeled section per lab. This is a take-home starter — keep it // and grow it into your team's identity query library. // HOW TO RUN // These run over Entra sign-in / audit logs. Two options: // - Log Analytics: if you export Entra logs to a workspace (Diagnostic settings -> // SigninLogs, AuditLogs), paste a query into Sentinel/Log Analytics -> Logs. // - Portal filters: if you do NOT export to a workspace, use the equivalent filters in // Entra -> Monitoring -> Sign-in logs / Audit logs (each section notes the filter). // The read-only PowerShell helpers (EKIDFUND-Discovery/Report) cover the same signals via Graph. // EXPECTED RESULTS // Small, explainable result sets you can read in class. Where a table/column may differ in // your tenant/cloud it is flagged // VERIFY. // NOTES // White-label. READ-ONLY — every query only reads logs; nothing changes the tenant. // Sign-in-log export to a workspace needs Entra ID P1/P2 + a Diagnostic setting — VERIFY in // your tenant. Column names evolve — VERIFY current schema (Logs -> schema). // ==========

How to run: // These run over Entra sign-in / audit logs. Two options: // - Log Analytics: if you export Entra logs to a workspace (Diagnostic settings -> // SigninLogs, AuditLogs), paste a query into Sentinel/Log Analytics -> Logs. // - Portal filters: if you do NOT export to a workspace, use the equivalent filters in // Entra -> Monitoring -> Sign-in logs / Audit logs (each section notes the filter). // The read-only PowerShell helpers (EKIDFUND-Discovery/Report) cover the same signals via Graph. // EXPECTED RESULTS // Small, explainable result sets you can read in class. Where a table/column may differ in // your tenant/cloud it is flagged // VERIFY. // NOTES // White-label. READ-ONLY — every query only reads logs; nothing changes the tenant. // Sign-in-log export to a workspace needs Entra ID P1/P2 + a Diagnostic setting — VERIFY in // your tenant. Column names evolve — VERIFY current schema (Logs -> schema). // ==========

EKIDFUND-Report.ps1

Purpose: A one-page, read-only report the instructor runs at the end of the kit: the cohort roster (members of Grp-EntraFund-Cohort), which lab sandbox objects exist, the Conditional Access lab policy state, MFA-registration coverage across the tenant (a fundamentals talking point), and the licensing tier. Optionally exports the rows to CSV for a training record. Read-only. HOW TO RUN pwsh -File .\EKIDFUND-Report.ps1 pwsh -File .\EKIDFUND-Report.ps1 -OutputCsv .\ekidfund-cohort.csv Consents to read-only scopes (Directory.Read.All, Policy.Read.All, Reports.Read.All, AuditLog.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary + optional CSV. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string]$OutputCsv, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: pwsh -File .\EKIDFUND-Report.ps1 pwsh -File .\EKIDFUND-Report.ps1 -OutputCsv .\ekidfund-cohort.csv Consents to read-only scopes (Directory.Read.All, Policy.Read.All, Reports.Read.All, AuditLog.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary + optional CSV. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string]$OutputCsv, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKIDFUND-Validate.ps1

Purpose: Turns 05-testing.md into a checklist run: confirms the cloud-only lab sandbox exists and is correctly named (Grp-EntraFund-LabTeam-Pilot, Grp-EntraFund-DynamicDept-Pilot, Lab-EntraFund-User1, App-EntraFund-LabApp-Pilot), that the Conditional Access lab policy CAFund-RequireMFA-LabTeam-ReportOnly exists in REPORT-ONLY with break-glass excluded, that the cohort group exists, and the detected licensing tier — then prints the knowledge-check Check-EntraFund-Pilot answer key for the instructor to score. Read-only. HOW TO RUN pwsh -File .\EKIDFUND-Validate.ps1 pwsh -File .\EKIDFUND-Validate.ps1 -BreakGlassUpn bg1@contoso.onmicrosoft.com,bg2@contoso.onmicrosoft.com Consents to read-only scopes (Directory.Read.All, Policy.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A list of PASS / FAIL / MANUAL gate lines and the knowledge-check answer key. Any FAIL = reinforce the mapped lab and re-check before declaring the cohort grounded. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string[]]$BreakGlassUpn, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: pwsh -File .\EKIDFUND-Validate.ps1 pwsh -File .\EKIDFUND-Validate.ps1 -BreakGlassUpn bg1@contoso.onmicrosoft.com,bg2@contoso.onmicrosoft.com Consents to read-only scopes (Directory.Read.All, Policy.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A list of PASS / FAIL / MANUAL gate lines and the knowledge-check answer key. Any FAIL = reinforce the mapped lab and re-check before declaring the cohort grounded. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string[]]$BreakGlassUpn, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')